Retail Banking Customer Segmentation Using K-Means Clustering: Feature Engineering with Snowflake–dbt and Python–pandas

Sunidhi Baroorkar
School of Computer Science and Engineering (SCOPE)
VIT-AP University
Amaravati, India
sunidhibaroorkar@gmail.com

***Abstract*—This study examines K-means customer segmentation using engineered features from a synthetic retail banking dataset and describes two data-processing implementations: Snowflake with dbt and local Python with pandas. Six CSV files represented customers, accounts, products, balances, and 300,769 combined transactions. The recorded Python customer mart contained 2,869 customers and 15 numerical features. In a new reproducible clustering analysis of this mart, features were standardized, K-means was fitted for $k = 2$–8 with 20 initializations per candidate, and $k = 5$ achieved the highest silhouette score (0.5426) on a fixed sample of 1,000 customers. The five clusters contained 414, 345, 826, 836, and 448 customers. A supplementary full-data silhouette check gave 0.5438 for $k = 5$ and 0.5429 for $k = 4$; ten random-initialization fits at $k = 5$ returned the same partition. A two-component principal component analysis (PCA) projection visualizes the assignments, while cluster profiles characterize differences in balance, transaction frequency, product holdings, and digital-channel use. The Snowflake–dbt implementation comprised 17 SQL models; a direct comparison of archived marts found matching customer identifiers and all 15 features within an absolute tolerance of 10^{-5}. In five previously recorded workflow executions, mean elapsed times were 25.416 s for Snowflake–dbt and 2.636 s for Python–pandas, measured across unequal execution boundaries. Three single-trial input-fault tests provide additional implementation observations. The clustering results describe this synthetic dataset; stability across alternative customer samples and predictive utility were not evaluated.**

***Index Terms*—customer segmentation, K-means, banking analytics, dbt, feature engineering, pandas, Snowflake**

I. Introduction

Customer segmentation groups banking customers by patterns in their balances, transaction activity, product use, and digital-channel behavior. Clustering depends on the quality and meaning of customer-level features assembled from multiple operational tables. This study investigates K-means clustering on an engineered retail banking feature mart, with the data preparation implemented in two ways: a 17-model Snowflake–dbt transformation graph and a local Python–pandas pipeline.

The primary question is: (RQ1) What customer groups are obtained by applying a documented K-means procedure to the 15-feature Python mart, and how do their descriptive profiles differ? Two supporting questions concern implementation: (RQ2) What workflow times were observed for the Snowflake–dbt and Python–pandas preparation paths? (RQ3) How did those paths respond to three injected input faults? The clustering analysis reported here was performed anew on the available Python mart. Previously recorded workflow times and fault tests are retained as separate observations, not presented as measurements of clustering execution.

The study contributes a reproducible segmentation analysis, a customer-level comparison of archived Snowflake and Python feature marts, and a bounded implementation evaluation. The segmentation findings do not establish generalization to real banking customers.

II. Background and Related Work

Dageville *et al.* describe Snowflake's separation of storage and compute and its shared-data cloud architecture [1]. That architecture provides a basis for warehouse execution, but it does not predict the elapsed time of a particular dbt project. dbt models express SQL transformations as dependent relations; dbt manages their execution order and materialization [2]. In contrast, pandas supplies tabular data structures and operations for local data processing. McKinney's original description emphasizes dataframe-based manipulation of structured data [3].

The TPC-DS benchmark specification defines a decision-support workload [4]. Unlike a standardized benchmark, the two workflows here have different timing boundaries. The reported values are consequently workflow timings, not a vendor-neutral comparison of computation engines. Pipino, Lee, and Wang frame data quality as a multidimensional assessment problem [5]; the present fault tests focus on a limited subset of input validity and completeness. Consequently, an observed successful run is not equivalent to verified data quality. In-process engines such as DuckDB [6] allow SQL execution locally, separating execution venue from programming language. Schelter *et al.* describe declarative data-quality checks [7]; such checks would be relevant to validating the outputs of the workflows studied here. These sources supply architectural and methodological context, not comparative results on the present dataset. For clustering, K-means minimizes within-cluster squared Euclidean distances and its output depends on feature

TABLE I
INPUT DATA AND RECORDED ROW COUNTS

Input file	Records
Customers	3,030
Accounts	3,000
Account balances	49,772
Products	9,376
Transactions, part 1	150,384
Transactions, part 2	150,385
Combined transactions	300,769

scaling and the selected cluster count [8]. Silhouette analysis measures separation relative to within-cluster cohesion [9].

III. DATASET AND IMPLEMENTATIONS

A. Source Data and Output

The study uses a synthetic retail banking dataset distributed across six CSV files and five data domains. Table I gives the source record counts. The two transaction files were combined into one logical transaction dataset. The raw-source quality audit reported 30 duplicate customer keys and missing cells in the customers (2,455), accounts (3,000), and products (28,899) files; these are source-level counts, not a causal attribution of downstream filtering. The source CSVs are not publicly distributed with this paper. The data are synthetic; no generation details are reported. The SQL pipeline used five corresponding Snowflake source tables. The recorded SQL and Python cleansing outputs each contained 291,802 transaction rows, a reduction of 8,967 (2.98%) from the combined input. Equal counts do not establish identical retained records. The final mart contained 2,869 customers, 161 (5.31%) fewer than the raw customer rows. Per-rule reductions were not recorded.

B. Snowflake–dbt Workflow

The cloud implementation uses 17 SQL models across validated, cleansed, intermediate, and final mart layers. Source records are read from Snowflake's BRONZE raw-landing tables; downstream models normalize and filter source fields, build customer-level aggregates, and assemble the analytical mart. The recorded configuration was dbt Core 1.12.5, dbt Snowflake adapter 1.12.1, four threads, and an X-Small virtual warehouse named `COMPUTE_WH`. Snowflake's documentation explains that virtual warehouse size determines the compute resources assigned to query execution [10]. The exact dbt timing command and timing source, warehouse resume state, and model materializations were not retained in the available experimental record.

C. Python–pandas Workflow

The Python script reads the six CSV files, concatenates the transaction partitions, standardizes categorical fields, converts identifiers and monetary amounts, filters inactive records, joins relational inputs, and computes customer-level metrics. The implementation fixes a reference date of August 1, 2026 and uses an explicit currency-to-INR rate mapping. The fixed

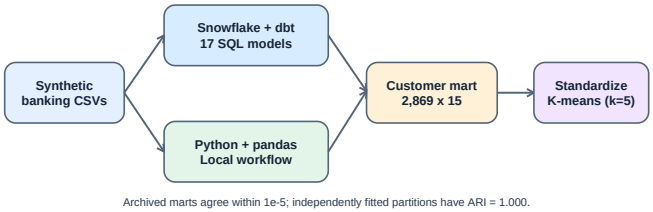


Fig. 1. End-to-end segmentation workflow. The two archived feature marts agree within 10^{-5}; clustering is independently reproduced on each export.

TABLE II
FIFTEEN FEATURES IN THE PYTHON CUSTOMER MART

`avg_monthly_balance`	`balance_volatility_ratio`
`balance_growth_rate`	`txn_frequency_per_month`
`active_months_ratio`	`days_since_last_txn`
`avg_txn_amount`	`txn_amount_cv`
`high_value_txn_ratio`	`net_cash_flow_ratio`
`debit_txn_share`	`account_tenure_months`
`num_active_products`	`overdraft_month_count`
`digital_channel_ratio`	

conversion constants, expressed as INR per unit of source currency, are INR 1.00, USD 87.25, EUR 94.60, GBP 111.30, and JPY 0.58. They are processing assumptions, not observed exchange rates. An unknown currency maps to a missing INR value in the Python code; equivalent SQL handling has not been established. The available project record reports transactions from February 1, 2025 through July 31, 2026, preceding the fixed reference date; the date bounds were checked against both supplied transaction CSVs. The script measures elapsed time with `time.perf_counter()` from before CSV loading through construction of the final dataframe. CSV export and metrics-file writing occur *after* the timer stops.

Fig. 1 summarizes the two paths and the downstream clustering stage; the archived marts were subsequently verified to have matching customer IDs and all 15 feature columns. The Python output features cover balance behavior (mean balance, balance volatility and growth, overdraft months), transaction activity (frequency, active-month ratio, recency, average amount, amount variability, high-value share, debit share, net cash-flow ratio, and digital-channel share), account tenure, and active product count. Table II gives the exact output column names found in the Python implementation. The script applies defaults and bounds to selected features, including a minimum customer-tenure denominator of one month. Both archived marts were inspected. Each contained 2,869 unique customer IDs and the same 15 feature columns. An identifier-keyed comparison found no numerical differences exceeding 10^{-5} across the 43,035 feature values (maximum absolute difference 5.0×10^{-7}).

D. Feature Definitions and Transformation Semantics

The Python implementation constructs a customer spine from eligible customer records and joins three intermediate metric groups: balance aggregates, transaction aggregates, and active-product counts. The eligibility rule retains active, nonstaff customers and active accounts. Only completed transactions

associated with eligible accounts enter transaction metrics. A fixed reference date of August 1, 2026 is used for account age and transaction recency. Amounts are converted to Indian rupees using fixed mapping constants for INR, USD, EUR, GBP, and JPY; these are experimental assumptions rather than historical or live foreign-exchange rates.

Table III records the operational definitions observed in the Python script. Here N is the number of included transactions, M is months since the customer-since date (minimum one), A counts distinct year–month transaction periods, c and d are the INR sums labeled credit and debit, and x is absolute converted transaction amount. The supplied source CSV amounts are nonnegative for both CREDIT and DEBIT records, as verified in both transaction partitions. The Python script sums these nonnegative amounts separately; therefore $(c - d)/(c + d)$ expresses net credit less debit divided by their sum, when the denominator is nonzero. A "high-value" transaction has converted amount above INR 100,000. Balance growth uses the average closing balances of the first and last three account-month rows ranked by balance date, with pandas first-occurrence tie handling; the selected rows can overlap when fewer than six observations exist and need not represent distinct calendar months.

These definitions describe the Python implementation. The archived SQL export agrees with its resulting feature values within an absolute tolerance of 10^{-5} on all matched customer records. This is output agreement for the archived exports, not proof that every intermediate transformation or exceptional input behaves identically.

E. Execution Environment and Reproducibility

Reproduction requires the synthetic input files, SQL models, Python implementation, execution configuration, and benchmark logs. The Python script and output were inspected; the SQL source models were documented in project records, and the archived SQL final mart was available for direct comparison. No public source-code repository, archived revision, or license is identified. The synthetic source data are not released with this manuscript. The archived Snowflake final mart was exported as a CSV, and its 15 feature columns were compared directly with the archived Python output. The original cloud benchmark commands and exact infrastructure state were not recovered.

F. SQL Model Dependencies

The recorded SQL model listing contains five validated models, five cleansed models, six intermediate models, and one mart model. The intermediate models build a customer profile, monthly balance observations, balance metrics, transaction base, transaction metrics, and product metrics. The final mart depends on the profile and three metric branches. The 17-model dependency structure is reconstructed from recorded SQL `ref()` calls rather than a deployed dbt documentation screenshot.

Fig. 2 depicts the recorded dbt model dependencies from validated and cleansed source domains through intermediate feature models to the customer-level mart. The arrows represent logical `ref()` dependencies, not physical database foreign-key constraints.

G. Downstream Dashboard Boundary

The downstream dashboard was outside the timing boundaries; no application response-time measurements were collected.

IV. K-Means Segmentation Method

The available Python customer mart was loaded from `mart_customer_ml_features.csv` (2,869 rows; 15 features plus `customer_id`). The identifier was excluded. All 15 features were numeric and contained no missing or infinite values. Each feature was standardized using its full-sample mean and population standard deviation (`StandardScaler`). The resulting matrix was clustered with scikit-learn K-means, Euclidean distance, `random_state=42`, `n_init=20`, and `max_iter=300`. Candidate cluster counts were $k = 2, \ldots, 8$; the selection criterion was the maximum silhouette coefficient evaluated on a fixed 1,000-row random sample (`random_state=42`). Inertia was recorded for the same candidate models as a complementary diagnostic.

The selected model was fitted to all 2,869 standardized observations. The same scaling and K-means settings were also applied independently to the archived Snowflake mart; the resulting partition was identical up to arbitrary label permutations (adjusted Rand index, ARI $= 1.000$). PCA with two components was then fitted to the standardized features solely for visualization; clustering itself used all 15 dimensions. Cluster sizes and unscaled per-feature means were computed for interpretation. These calculations were executed as a new analysis of the archived Python output, using Python libraries scikit-learn 1.8.0, pandas 2.2.3, and NumPy 2.3.5. These are the clustering-analysis environment versions, not necessarily those of the earlier pipeline benchmarks. The source mart, analysis script, and random seeds are required to reproduce the cluster labels; no external ground-truth segment labels were available. A separate verification script evaluated full-data silhouette scores for $k = 4$ and $k = 5$, ten random-initialization fits for $k = 5$, and pairwise Pearson correlations between the 15 input features. The same full dataset and feature scaling were used for each seed, so this checks initialization sensitivity rather than sampling stability.

V. Experimental Design

A. Timing Protocol

Five elapsed-time observations were collected for each implementation. The Snowflake measurement represents execution of the dbt workflow with data already present in the warehouse. It includes orchestration and remote execution but excludes initial upload and ingestion of the raw CSVs. The Python measurement begins before local CSV reads and ends after dataframe construction, excluding persistence of the output. Consequently, the two times are not strictly matched end-to-end ingestion-to-output durations, nor are they isolated computation times. The known configuration and unavailable benchmark

TABLE III
OPERATIONAL DEFINITIONS OF THE 15 PYTHON CUSTOMER-LEVEL FEATURES

Feature	Calculation or interpretation
`avg_monthly_balance`	Mean of account-month average balance observations associated with the customer.
`balance_volatility_ratio`	Sample standard deviation of account-month average balances divided by absolute mean; special rule sets 3 when absolute mean is below 1,000; bounded to [0,3].
`balance_growth_rate`	(Mean last-three ranked closing balances − mean first-three ranked closing balances) / (absolute first-three mean +1); bounded to [−1,3].
`txn_frequency_per_month`	N/M.
`active_months_ratio`	A/M; final value bounded to [0,1].
`days_since_last_txn`	Calendar days between the fixed reference date and the latest included transaction; default 730 if missing.
`avg_txn_amount`	Mean of x across included transactions.
`txn_amount_cv`	Sample standard deviation of x divided by mean x; zero if undefined.
`high_value_txn_ratio`	Share of included transactions with converted amount $> 100{,}000$ INR; bounded to [0,1].
`net_cash_flow_ratio`	$(c-d)/(c+d)$ where defined; final value bounded to [−1,1].
`debit_txn_share`	Share of included transactions labeled debit; default 0.5 if unavailable.
`account_tenure_months`	Calendar-month difference from customer-since date to reference date, with minimum one month.
`num_active_products`	Count of distinct active product codes for a customer; default zero.
`overdraft_month_count`	Count of account-month balance rows with overdraft flag Y; not necessarily distinct calendar months.
`digital_channel_ratio`	Share of included transactions with channel MOBILE_APP or WEB; bounded to [0,1].

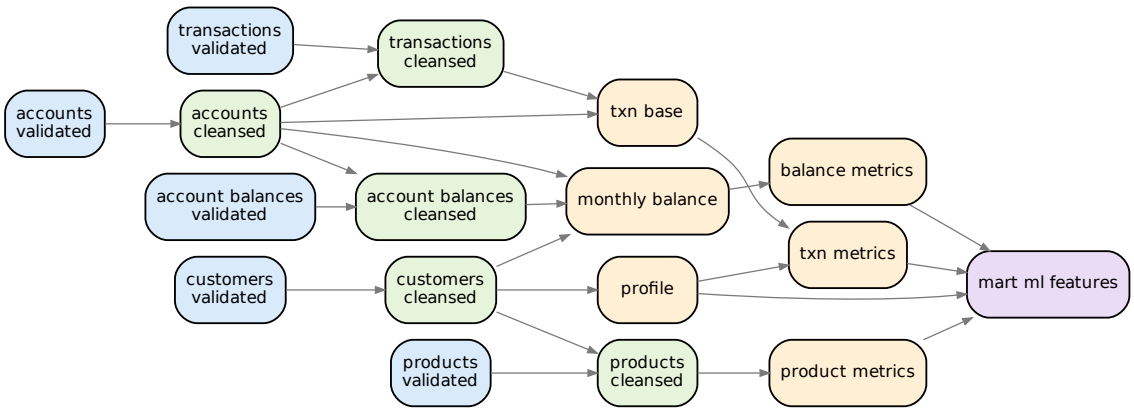


Fig. 2. Logical dbt dependency graph reconstructed from the recorded SQL model definitions. Arrows indicate declared `ref()` dependencies.

TABLE IV
RECORDED ENVIRONMENT AND OUTSTANDING DETAILS

Item	Recorded setting / missing detail
Cloud	dbt Core 1.12.5; Snowflake adapter 1.12.1; 4 threads; X-Small COMPUTE_WH
Local	Single Python process; package versions unrecorded
Hardware	CPU, RAM, disk, and OS unrecorded
Cache / order	Cache and resume states unrecorded; run labels 1–5
Boundaries	Cloud: recorded dbt workflow, no initial ingestion; local: CSV read through dataframe, no output writing

details are summarized in Table IV. Run labels 1–5 do not establish independently verified chronological order.

Let $t_{i,j}$ denote elapsed time for implementation i in run j. We report the arithmetic mean $\bar{t}_i = \frac{1}{5}\sum_{j=1}^{5} t_{i,j}$ and the ratio $R = \bar{t}_{\text{cloud}}/\bar{t}_{\text{local}}$. The ratio is descriptive of these two observed workflows and should not be interpreted as a hardware-normalized speedup. The median, minimum, maximum and range are also reported. Cache and warehouse state were not controlled; no inferential test or confidence interval is presented. Sample standard deviations (using denominator $n-1$) are reported to describe variability, not to establish statistical significance.

B. Resource Observation and Fault Injection

The separate Python memory measurement is reported with the results in Section VII-C. No comparable cloud memory, credit consumption, or cost was measured.

Three fault scenarios were applied to isolated test copies of the source data: replacing one transaction amount with a non-numeric string, duplicating a customer row, and replacing one transaction amount with NULL. Each scenario was run once per implementation. The cloud tests used a separate test schema and test-prefixed outputs; the original BRONZE source tables were not changed. The source-row identifier, downstream eligibility of the altered transaction, and exact representation

TABLE V
K-MEANS SELECTION ON THE PYTHON MART

k	Inertia	Sample silhouette
2	28074.03	0.4047
3	14693.33	0.5170
4	8953.06	0.5411
5	6816.77	**0.5426**
6	6301.03	0.4608
7	5927.66	0.4616
8	5666.89	0.4136

of the injected NULL were not preserved in the test record. For each case, the recorded outcome was whether execution failed or completed and, where available, the resulting row counts. Completion was not taken as proof that all derived values remained correct.

VI. CUSTOMER SEGMENTATION RESULTS

A. Cluster Selection and Visualization

Table V lists the recorded inertia and sampled silhouette scores. The largest sampled silhouette was 0.5426 at $k = 5$, narrowly above $k = 4$ (0.5411). The small difference means the selection should not be interpreted as evidence that five segments are uniquely optimal. The inertia and silhouette diagnostics are reported numerically in Table V. The two PCA components explain 40.41% and 32.99% of standardized-feature variance, respectively (73.40% combined). Fig. 3 reproduces the author's original five-segment PCA visualization. Its named legend is retained as supplied; the archived numerical results are reported using cluster identifiers 0–4 because the name-to-identifier correspondence was not included in the verification artifacts. Apparent separation in a two-dimensional projection does not establish separation in the original 15-dimensional space.

B. Additional Cluster Validation

To assess sensitivity to random initialization, K-means with $k = 5$ and 20 initializations was refitted using seeds 0, 1, 2, 3, 7, 11, 21, 42, 73, and 99. All ten partitions agreed with the reference seed-42 fit (adjusted Rand index, ARI $= 1.000$ in each comparison), and their inertias matched to the displayed precision. This is evidence of repeatability across these seeds on the *same* standardized dataset, not of stability under resampling or temporal change.

A full-data silhouette calculation for $k = 4$ and $k = 5$ yielded 0.5429 and 0.5438, respectively. The difference (0.0009) is smaller than the sampled-score difference (0.0015), and neither establishes that five clusters are uniquely optimal. Feature-correlation screening found $|r| = 0.9927$ between average transaction amount and high-value transaction ratio, $|r| = 0.9284$ between balance volatility and overdraft-month count, and $|r| = 0.9272$ between net cash-flow ratio and debit share. Standardization does not remove these dependencies; the distance-based grouping may give correlated feature families disproportionate weight. No feature-ablation or temporal validation was performed.

C. Segment Sizes and Descriptive Profiles

The five clusters contain 414, 345, 826, 836, and 448 customers (clusters 0–4). Table VI reports selected unscaled means, showing heterogeneous balance levels, transaction activity, product holdings, and digital-channel usage. Cluster 0 has low average balance and activity; cluster 1 has the highest mean balance; clusters 2 and 3 have comparatively high digital-channel ratios; and cluster 4 has the highest mean active-product count. These are descriptive differences, not validated business categories. Table VI shows the membership distribution.

D. Cross-Pipeline Feature and Cluster Agreement

The archived Snowflake customer mart and the Python customer mart were aligned by `customer_id` after normalizing column-name case. Both contained 2,869 distinct customers and the same 15 feature columns; no customer identifiers were unmatched. Across 43,035 paired feature values, the largest absolute difference was 5.0×10^{-7} and none exceeded a comparison tolerance of 10^{-5}. Independently standardizing each export and fitting $k = 5$ K-means with `random_state=42`, `n_init=20`, and `max_iter=300` yielded identical partitions (ARI $= 1.000$). These checks establish numerical and clustering agreement for the two archived outputs; they do not test equivalence of intermediate records or edge-case handling. Table VII summarizes the direct comparison.

VII. SUPPORTING PIPELINE EVALUATION

A. Observed Workflow Time

Table VIII reports the five recorded runs. The mean Snowflake–dbt duration was 25.416 s; the mean Python–pandas duration was 2.636 s. The ratio of means was 9.64. The sample standard deviations were 2.725 s (Snowflake–dbt) and 0.759 s (Python–pandas). Recorded ranges were 22.71–29.24 s and 1.9326–3.7821 s, respectively. Python had the shorter recorded time in every run. Table VIII shows the individual measurements, making the run-to-run spread visible. The run labels are identifiers rather than a verified chronology; no cache-effect interpretation is possible.

B. Separate Partial Python Baseline

A separately supplied Python script measured a *partial* transaction-cleaning and customer-aggregation workflow, not the full 17-model dbt-equivalent feature pipeline. It read the two transaction CSVs and the customer and account tables before starting its timer; each timed run copied and processed a prefix of 10,000, 100,000, or 300,769 transaction records. Two runs were recorded at each size, with medians of 0.1236, 0.6820, and 2.2458 s, respectively. The output contained 2,117, 2,619, and 3,000 aggregated customers. Because data loading was outside the timed section, the output schema differs from the final mart, and no corresponding Snowflake size-scaling runs exist, these results are presented as an independent local processing observation and *not* as a cloud-versus-local scaling benchmark.

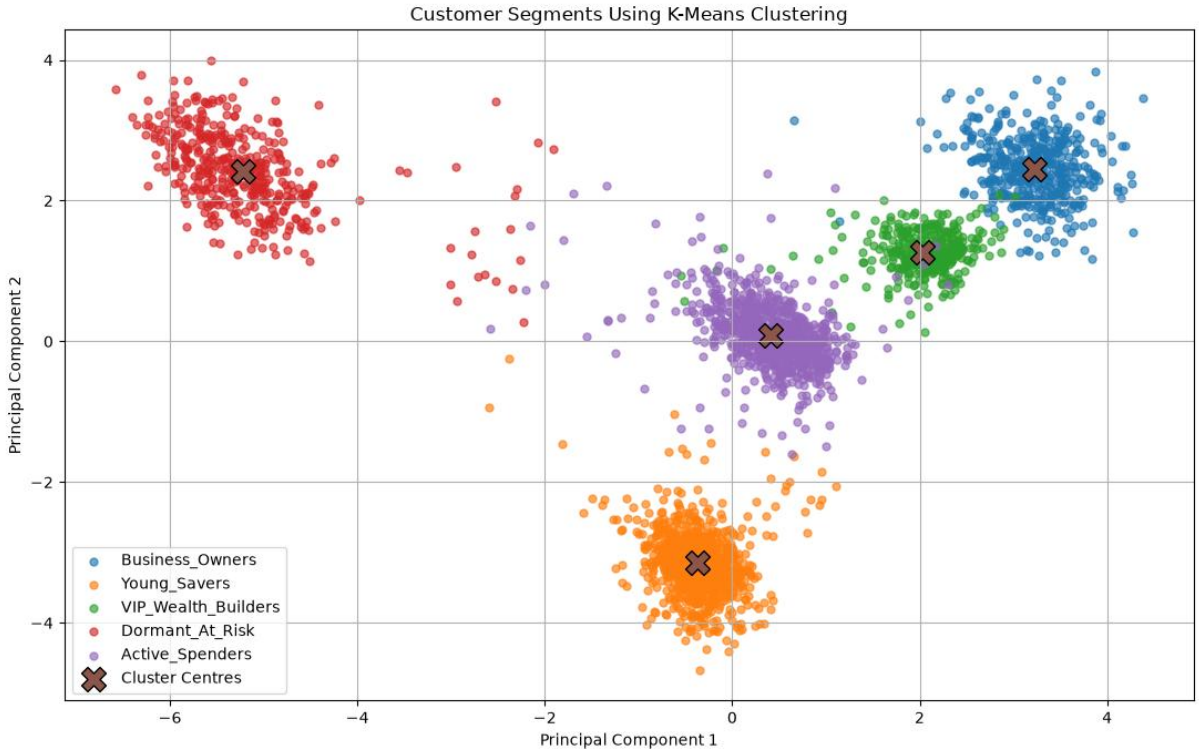


Fig. 3. Original customer-segmentation PCA visualization supplied by the author. Colors denote the author's descriptive groups (Business Owners, Young Savers, VIP Wealth Builders, Dormant At Risk, and Active Spenders); crosses denote plotted centers. A verified mapping to the independently refitted cluster identifiers 0–4 is unavailable.

TABLE VI
SELECTED CUSTOMER-FEATURE MEANS BY K-MEANS CLUSTER

Cluster	Customers	Mean monthly balance (INR)	Transactions/month	Active products	Digital-channel ratio
0	414	13,899.27	0.27	1.00	0.30
1	345	3,772,760.28	0.87	4.16	0.21
2	826	142,574.69	2.27	1.25	0.94
3	836	222,702.34	4.16	2.95	0.87
4	448	2,436,134.70	2.96	6.14	0.54

TABLE VII
ARCHIVED MART AND CLUSTER COMPARISON

Check	Observed result
Customers in each mart	2,869
Matching customer IDs	2,869
Matched features	15
Paired numeric values	43,035
Maximum absolute difference	5.0×10^{-7}
Differences $> 10^{-5}$	0
K-means partition ARI	1.000

TABLE VIII
RECORDED WORKFLOW TIMES (SECONDS)

Run	Snowflake + dbt	Python + pandas
1	26.78	3.7821
2	22.99	3.0057
3	22.71	2.3472
4	25.36	1.9326
5	29.24	2.1135
Mean	25.416	2.636
Median	25.36	2.347
Minimum	22.71	1.9326
Maximum	29.24	3.7821
Range	6.53	1.8495
Sample SD	2.725	0.759

C. Resource Observation

A separate Python run took 3.5902 s and recorded peak resident set size (RSS) of 304.03 MB. This run was not part of the five-run timing series; the measurement utility, operating system, and selection procedure were not retained. No comparable cloud measurement is available.

D. Data-Quality Fault Responses

Table X summarizes the fault-injection results. For the non-numeric-amount case, the injected value `INVALID_AMOUNT`

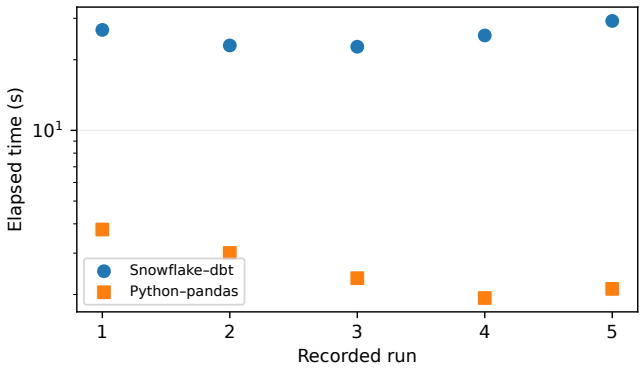


Fig. 4. Per-run workflow latency (logarithmic vertical axis). Timed stages and execution environments differ.

TABLE IX
SEPARATE PARTIAL PYTHON BASELINE (SECONDS)

Input rows	Run 1	Run 2	Median
10,000	0.1371	0.1101	0.1236
100,000	0.7955	0.5686	0.6820
300,769	2.6484	1.8433	2.2458

caused Python's strict numeric conversion to raise `ValueError`; Snowflake rejected the value during a numeric cast with recorded error code 100038. Both therefore failed visibly on this specific malformed input.

For the duplicate-customer case, customer 4568 appears in the available baseline Python mart, and in the archived SQL mart. Adding a second record for customer 4568 increased the raw customer count from 3,030 to 3,031. Both workflows produced 2,869 final customer rows. The SQL test completed all 17 models. The unchanged row counts are consistent with deduplication or filtering, but cannot establish whether the injected customer was eligible, which record survived, or whether feature values were equal.

For the missing-amount case, Python failed when converting a nullable Boolean-derived high-value indicator to `int64`; the exception class was not retained in the provided records. This was a script-specific indicator-conversion failure rather than a general pandas inability to handle missing values. The recorded SQL test completed all 17 models in 25.39 s, and the NULL was observed in the validated transaction table. However, no independent audit of downstream feature values was recorded. The strongest supported conclusion is that the SQL workflow *completed with this input*, not that it correctly imputed or propagated missing amounts throughout the mart.

VIII. DISCUSSION AND VALIDITY LIMITATIONS

The primary finding is a five-group partition of the 2,869-customer feature mart, with sampled silhouette 0.5426. The archived Snowflake and Python outputs have the same 2,869 customer identifiers and 15 feature columns, with maximum absolute feature difference 5.0×10^{-7}; independently refitted K-means models yield ARI $= 1.000$. Cluster means show differences in account balances, activity, products, and digital use, but the small silhouette difference between $k = 4$ and $k = 5$ warrants caution. The clusters were evaluated on one synthetic dataset with no external labels, sample-resampling study, temporal holdout, or downstream decision-outcome assessment. Ten initialization-seed fits agreed exactly, but the observed feature correlations and close $k = 4/k = 5$ silhouettes limit claims of uniquely determined segment structure. The PCA view is descriptive. Snowflake–dbt supplied a modular 17-model preparation architecture, and the archived Snowflake mart reproduced the five-way partition. This verifies output agreement for these exports, not general correctness under all inputs.

TABLE X
OBSERVED RESPONSES TO THREE INJECTED FAULTS

Fault	Python + pandas	Snowflake + dbt
Non-numeric amount	`ValueError`; rejected	Numeric-cast error; rejected
Duplicate customer	Completed; 2,869 final rows	17/17 models; 2,869 final rows
NULL amount	Failed on nullable-indicator conversion	17/17 models; output accuracy unverified

The local pandas workflow had lower recorded elapsed time in all five observations, but the result is not an engine-level performance comparison. Snowflake ran in a remote managed service with dbt orchestration and network interaction; pandas ran locally in one Python process. The initial cloud ingestion was excluded, whereas local CSV loading was included. The persistence and export stages were not matched. These mismatches prevent attributing the 9.64 ratio solely to SQL versus dataframe computation. The modest input and 17-model execution graph may also affect the relative weight of orchestration overhead; its exact contribution was not isolated experimentally. Run-to-run variability is visible, but the five observations do not establish steady-state behavior or confidence in a cross-platform speed ranking.

The NULL-amount test produced different execution outcomes, but the downstream SQL features were not checked. Successful execution and unchanged row counts do not establish feature correctness. Only five runs per workflow and one dataset size were measured, without comparable cloud memory or cost observations. No significance test was performed.

The main threats to validity are unequal timing boundaries, undocumented cache and hardware conditions, and possible sensitivity to changes in source records and transformation rules. The most direct follow-up is matched-boundary timing with recorded configurations, alongside feature-ablation and customer-resampling checks.

IX. DATA AND CODE AVAILABILITY

The synthetic source CSVs, archived feature marts, and implementation code are not publicly distributed with this manuscript. The private reproduction package contains the two archived feature marts, clustering scripts, cluster assignments, and aggregate metrics. Reproducing source-to-mart transforma-

tions also requires the original synthetic input files and SQL models. The archived Snowflake final mart and the Python customer mart are retained in a private reproduction package; neither is publicly distributed. The original cloud timing commands and original fault-test traceback were not available for independent re-execution. These restrictions prevent full replication of the dual-pipeline experiment from the package alone.

X. Conclusion

A documented K-means analysis of 2,869 customers and 15 engineered banking features selected five clusters by the highest sampled silhouette score (0.5426 among $k = 2$–8). The clusters differ in balance levels, transaction frequency, active products, and digital-channel ratios; the first two PCA components explain 73.40% of standardized-feature variance. Ten random-initialization fits gave identical $k = 5$ partitions, but the closely competing $k = 4$ silhouette and correlated features limit stronger conclusions. These results remain descriptive for one synthetic dataset and require resampling and external-utility assessment before practical interpretation. The Snowflake–dbt preparation workflow organized transformations into 17 dependent SQL models, and its archived customer mart reproduced the Python-derived clustering partition (ARI $= 1.000$; maximum feature discrepancy 5.0×10^{-7}). Previously recorded workflow timings favored local Python in elapsed seconds, but incomparable timing boundaries prevent a platform-level speed conclusion. The observed agreement is limited to the archived exports and does not establish equivalent performance or fault handling.

References

[1] B. Dageville *et al.*, "The Snowflake Elastic Data Warehouse," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2016, pp. 215–226, doi: 10.1145/2882903.2903741.

[2] dbt Labs, "SQL models," *dbt Developer Hub*. Accessed: Oct. 29, 2026. [Online]. Available: https://docs.getdbt.com/docs/build/sql-models.

[3] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.

[4] Transaction Processing Performance Council, "TPC-DS," Accessed: Oct. 30, 2026. [Online]. Available: https://www.tpc.org/tpcds/.

[5] L. L. Pipino, Y. W. Lee, and R. Y. Wang, "Data quality assessment," *Commun. ACM*, vol. 45, no. 4, pp. 211–218, 2002, doi: 10.1145/505248.506010.

[6] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2019, pp. 1981–1984, doi: 10.1145/3299869.3320212.

[7] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endow.*, vol. 11, no. 12, pp. 1781–1794, 2018, doi: 10.14778/3229863.3229867.

[8] scikit-learn developers, "KMeans," *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html.

[9] scikit-learn developers, "Silhouette analysis for KMeans clustering," *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html.

[10] Snowflake, "Overview of warehouses," *Snowflake Documentation*. Accessed: Oct. 29, 2026. [Online]. Available: https://docs.snowflake.com/en/user-guide/warehouses-overview.